REP-005 · PLAN-004 proposed: fix the rejoin problem in online play

Game	Fruit Market
Report date	2026-10-26 14:20 (Asia/Kuala_Lumpur)
Build	`main @ 935a287` (live game still 1.1.0, `c915e49`)
Type	Planning only. **No game implementation changes were made.** Read-only code inspection only.
Approval status	PLAN-004 **awaiting approval** (DEC-010, DEC-011)

1 · The problem

Sometimes, after a player rejoins an online room, the game screen doesn't come back:

CI run	Situation	Waited	Result
Run 1 (`da2eea9`)	Second player joins via Quick match	15 s	Game screen never appeared
Run 4 (`63c3682`, docs only)	Player A reloads the page mid-match	20 s	Game screen never appeared
Runs 2, 3 and 5, plus 8+ local runs	Same steps	—	Passed (usually about 1 s)

There was no game-code change between the failing and passing runs, so the problem is timing-dependent. Real players can trigger it: phones reload pages when memory is low, and mobile connections drop. The tests saved **no evidence** when it happened: the local server's log was discarded, and the online test takes no screenshot on failure.

2 · What the code shows (read-only)

How rejoining works today:

1. The phone keeps a seat token per room.
2. On reload, the online screen sees `?room=CODE` plus the token and reconnects.
3. The server's `hello` matches the token to the seat and sends the room state.
4. The game screen appears once the state says the match has started.

Hypotheses, none confirmed yet:

#	Hypothesis	Where	Fits run 1	Fits run 4	Evidence so far
H1	In a quick-match lobby, a seat is deleted when its connection closes. If the waiting player's connection	`server/room.ts closed()`	Yes	No	Plausible from code

#	Hypothesis	Where	Fits run 1	Fits run 4	Evidence so far
	blips just as the stranger joins, the match never starts				
H2	The old connection's "closed" event reaches the server *after* the new `hello`, wrongly marking the seat offline or freeing it	`server/room.ts` `closed()` / `hello()`	Yes	Yes	Plausible; order depends on server timing
H3	The server answers the rejoin with a non-fatal error and the screen stays on "Connecting…" with no retry	`server/room.ts`, `src/ui/Online.tsx`	Maybe	Yes	No error is shown or logged in the test
H4	The client drops or ignores the first room state after reconnecting (version check or listener timing)	`src/net/client.ts`, `useGameController.ts`	Maybe	Yes	Code looks right, but it isn't tested under slow timing
H5	Only the test environment: an overloaded CI machine is slower than the timeout	Test scripts	Yes	Yes	Normal time is about 1 s, so 15–20 s would be extreme. Not assumed without data

Ruled out: a seat being lost when the server sleeps. Seats are stored on each connection in a way that survives the server sleeping (`serializeAttachment`).

3 · PLAN-004 · Reconnect fix (evidence first)

Player problem: a player who reloads or loses connection can be stuck on a waiting screen instead of returning to their match.

Included scope (small steps)

1. **Evidence.** When the online or quick-match test fails, it saves:
   - both players' screens, page text and browser console
   - the room messages each player received
   - the local server log, kept as a CI artifact for 7 days
2. **Reproduce.** A new stress test, `npm run test:reconnect`, repeats three kinds of cycle and times how long the game screen takes to return:
   - reload mid-match
   - a connection drop (the browser goes offline, then online)
   - quick-match pairing

   It runs normally and with the CPU slowed 4× (like a cheap phone). It can be run locally and from a manual CI workflow (DEC-011). I'll run it in batches (target 300 cycles) and record the failure rate.
3. **Isolate.** Compare failing and passing cycles to confirm or reject H1–H5.

4. **Fix the measured cause** with the smallest change, plus a regression test that fails before the fix and passes after.
    - If the cause is in seat or token handling (who owns a seat), a fresh, separate review checks the change, because that's where security matters.
    - If the cause is only test timing (H5), the fix goes in the tests, but only with stress-test data showing real rejoins work.
5. **Prove.** The stress test and CI must stay green repeatedly (acceptance below).

Excluded

- Rules, balance, `RULES_VERSION` and saves.
- Message-format changes that old app versions can't understand (if one is needed, I stop and replan).
- Publishing (that needs "PREPARE RELEASE" and your Deploy).
- Package C.
- Making tests "pass" by just waiting longer or retrying.

Roles: network/backend engineer and QA (lead), technical lead, plus security review if seat or token logic changes.

Likely files:

- `scripts/e2e-online.cjs`, `scripts/e2e-quick.cjs`, `scripts/e2e.mjs` (diagnostics, server log)
- new `scripts/e2e-reconnect.cjs`
- new `.github/workflows/reconnect-stress.yml` (manual)
- `package.json`
- then, depending on the cause: `server/room.ts` or `src/net/client.ts` or `src/ui/Online.tsx`, plus a test in `tests/room.test.ts`

Acceptance (observable)

- R1. `npm run test:reconnect` prints time-to-game for every cycle, the failures, and the 95th percentile.
- R2. The root cause is named with evidence from a failing cycle (a log or message excerpt). If it isn't reproduced after at least 300 cycles, I report **NOT REPRODUCED** with the conditions tried, and stop rather than guess.
- R3. If reproduced: the failure rate before the fix is recorded, and after the fix at least 100 slowed-down cycles have 0 failures.
- R4. A deliberately forced failure in the online test produces the diagnostics files and the server log artifact.
- R5. A regression test fails on the old code and passes on the fixed code.
- R6. `verify:full` is green on 3 consecutive CI runs; the leak check stays at 0; all unit tests stay green; no rules or save changes.

Checks: R1–R6, `verify`, the manual stress workflow, and CI.

Cost and data: RM0. GitHub Actions is free for public repositories. Diagnostics contain only test players and expire after 7 days.

Risks

- It may not reproduce outside CI. Mitigation: the slowed-down mode and the CI workflow, and a hard stop at 300 cycles with a report.
- A fix in seat handling could affect reconnect security. Mitigation: a regression test, the existing room tests, and a separate review.
- Time: likely 1–2 sessions. I'll report at the end of step 3 if the cause turns out larger than a small fix.

Rollback: revert the package commits. Players are unaffected until a release is published.

How you'll see the result: once the fix is on the preview, reload the page mid-match on your phone, or switch to airplane mode for 5 seconds and back. The match should return within a few seconds.

4 · Decisions

ID	Question	Recommendation	Status
DEC-010	How to fix the rejoin problem	Evidence and reproduction first, then fix only the measured cause (PLAN-004)	Awaiting approval
DEC-011	Run the stress test on every push?	No: on demand (manual workflow and local runs)	Awaiting approval

5 · Still open

- Pending decisions from earlier: DEC-001, DEC-004, DEC-005, and PLAN-003 (outline only).
- Your preview playtest, branch protection, your phones and your Cloudflare plan.